Data Science with Python

Data Science Overview

Learning Objectives

By the end of this lesson, you will be able to:

- Define Data Science
- Discuss the roles and responsibilities of a Data Scientist
- List various applications of Data Science
- Explain Data Science importance
- Describe Python and its importance

What Is Data Science?

Some common definitions of Data Science are:

A powerful new approach to make discoveries from data

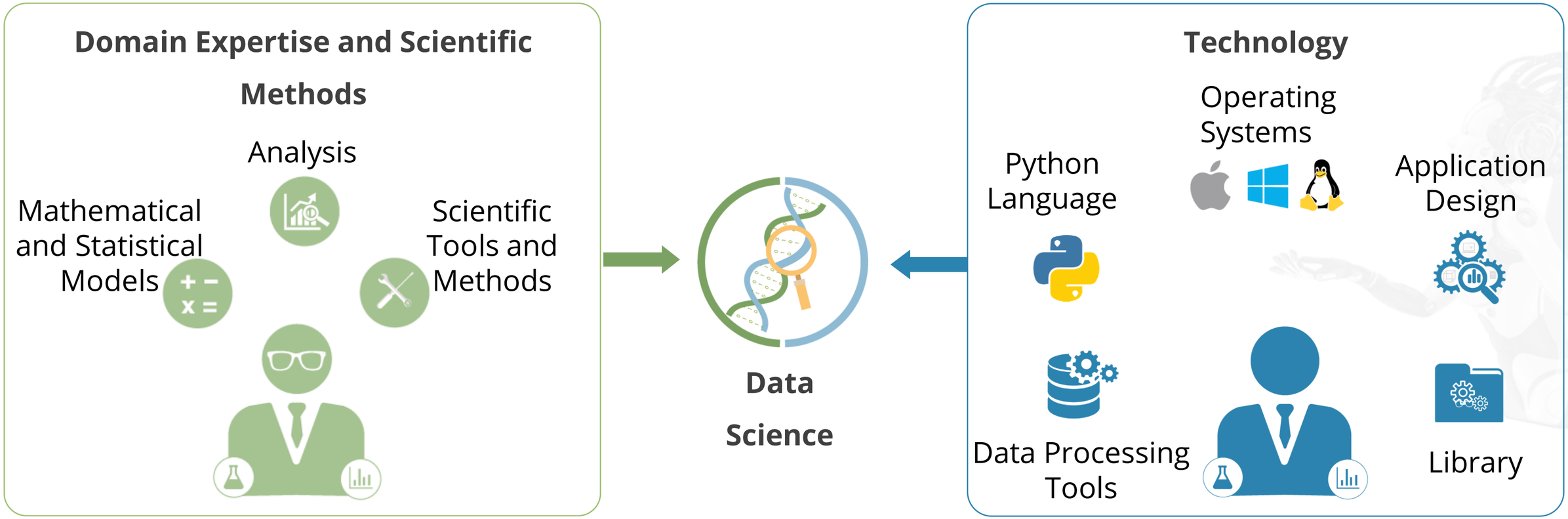
An automated way to analyze enormous amounts of data and extract information

Data Science

A new discipline that combines the aspects of statistics, mathematics, programming, and visualization to turn data into information

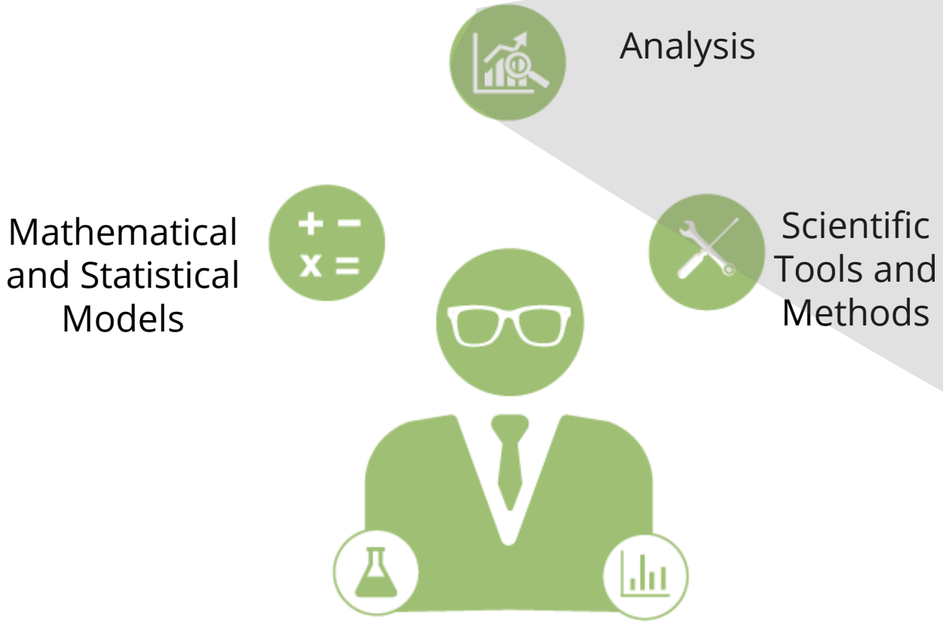
Components of Data Science

When you combine domain expertise and scientific methods with technology, you get Data Science.



Domain Expertise and Scientific Methods

Data Scientists collect, explore, analyze, and visualize data. They apply mathematical and statistical models to find patterns and solutions in the data.



Data analysis can be:

- Descriptive: Study a dataset to decipher the details
- Predictive: Create a model based on existing information to predict outcome and behavior
- Prescriptive: Suggest actions for a given situation using the collected information

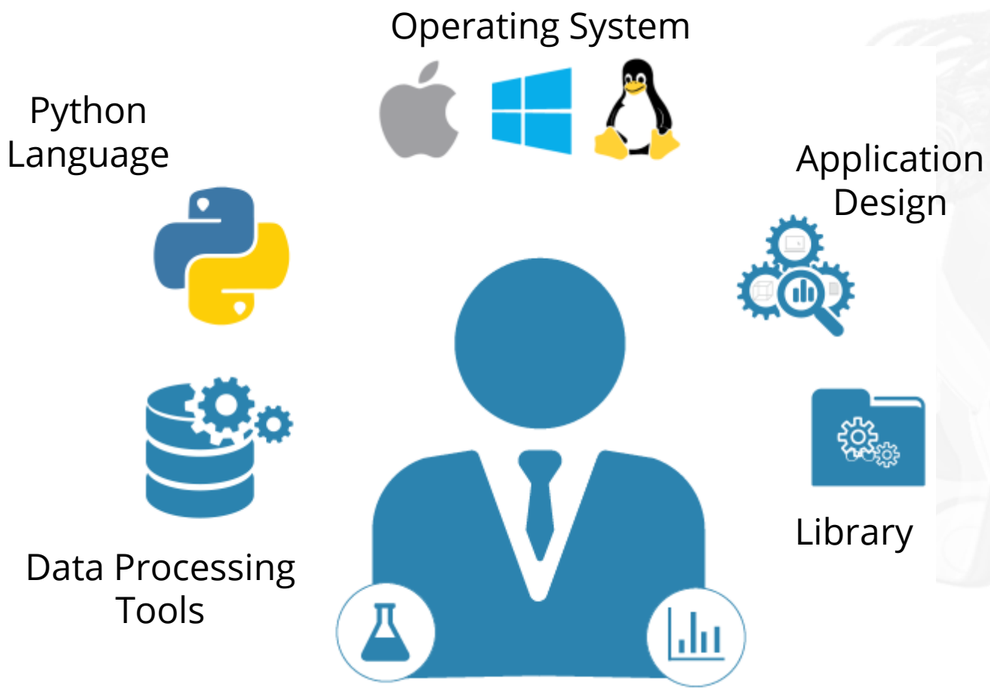
Data Processing and Analytics

Modern tools and technologies have made data processing and analytics faster and efficient.

These technologies help Data Scientists to:

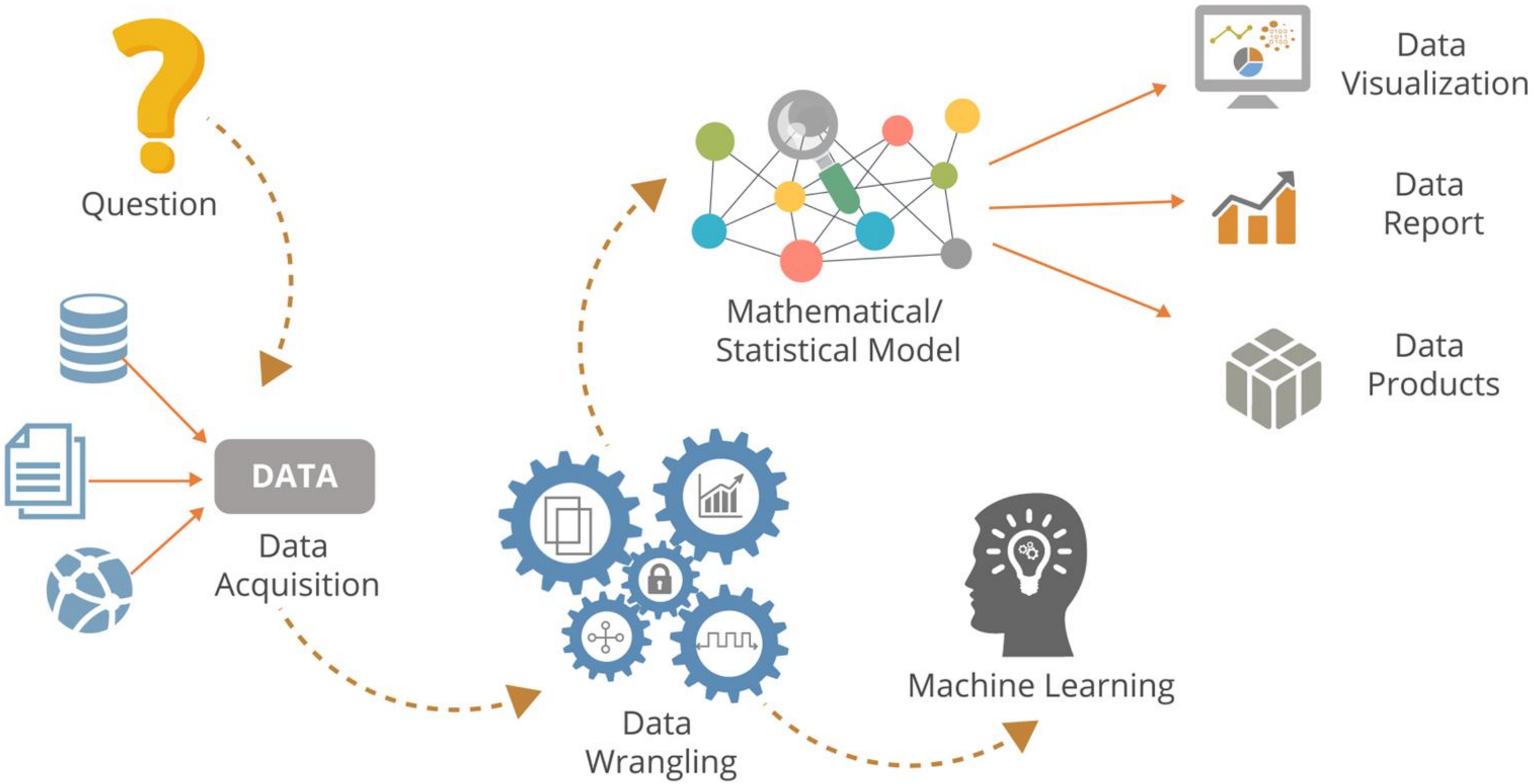
- Build and train machine learning models
- Manipulate data with technology
- Build data tools, applications, and services
- Extract information from data

Technology



Data analysis that uses only technology and domain knowledge without mathematical and statistical knowledge often leads to incorrect patterns and wrong interpretations.
This can cause serious damage to businesses.

Role of a Data Scientist



Basic Skills of a Data Scientist

A Data Scientist should be able to:

- Ask the right questions
- Understand data structure
- Interpret and wrangle data
- Apply statistical and mathematical methods
- Visualize data and communicate with stakeholders
- Work as a team player

Sources of Big Data

Data Scientists work with different types of datasets for various purposes. Now that Big Data is generated every second through different media, the role of Data Science has become more important.

3 Vs of Big Data

Volume	Enormous amount of data generated from various sources
Velocity	Large amount of data streaming in at great speeds, which requires quick data processing
Variety	Different formats of data: Structured, Semi-structured, and Unstructured

Big Data is a huge collection of data stored on distributed systems/machines popularly referred to as Hadoop clusters.

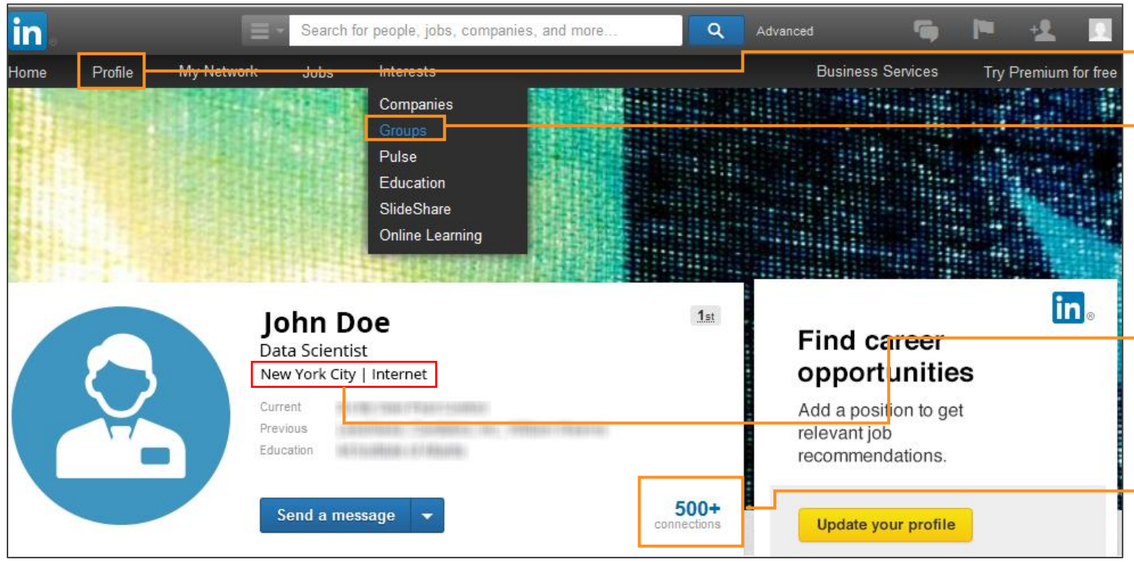
Data Science helps extract information from the data and build information-driven enterprises.

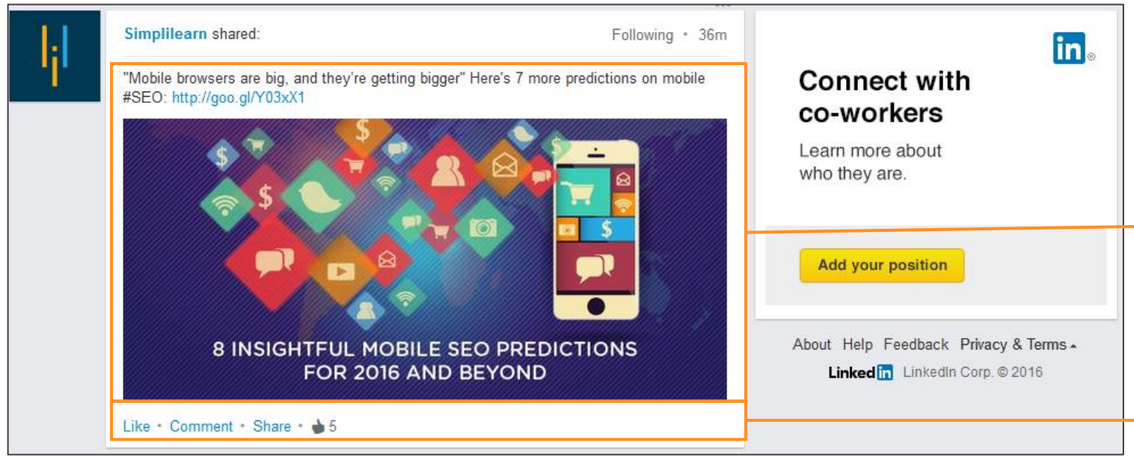
Different Sectors Using Data Science

Various sectors use Data Science to extract the information they need to create different services and products.

Using Data Science: Social Network Platforms

LinkedIn uses data points from its users to provide them with relevant digital services and data products.





Profil
e

Groups

Locatio
n

Connections

Post

Likes

Data Points

Information

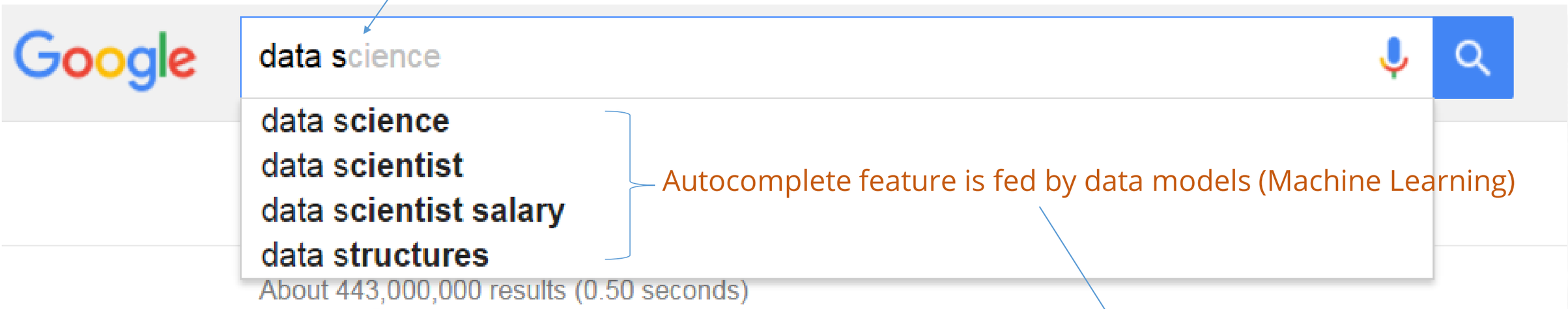
Digital Services

Data Products

Using Data Science: Search Engines

Google uses Data Science to provide relevant search recommendations as the user types a query.

Search keyword



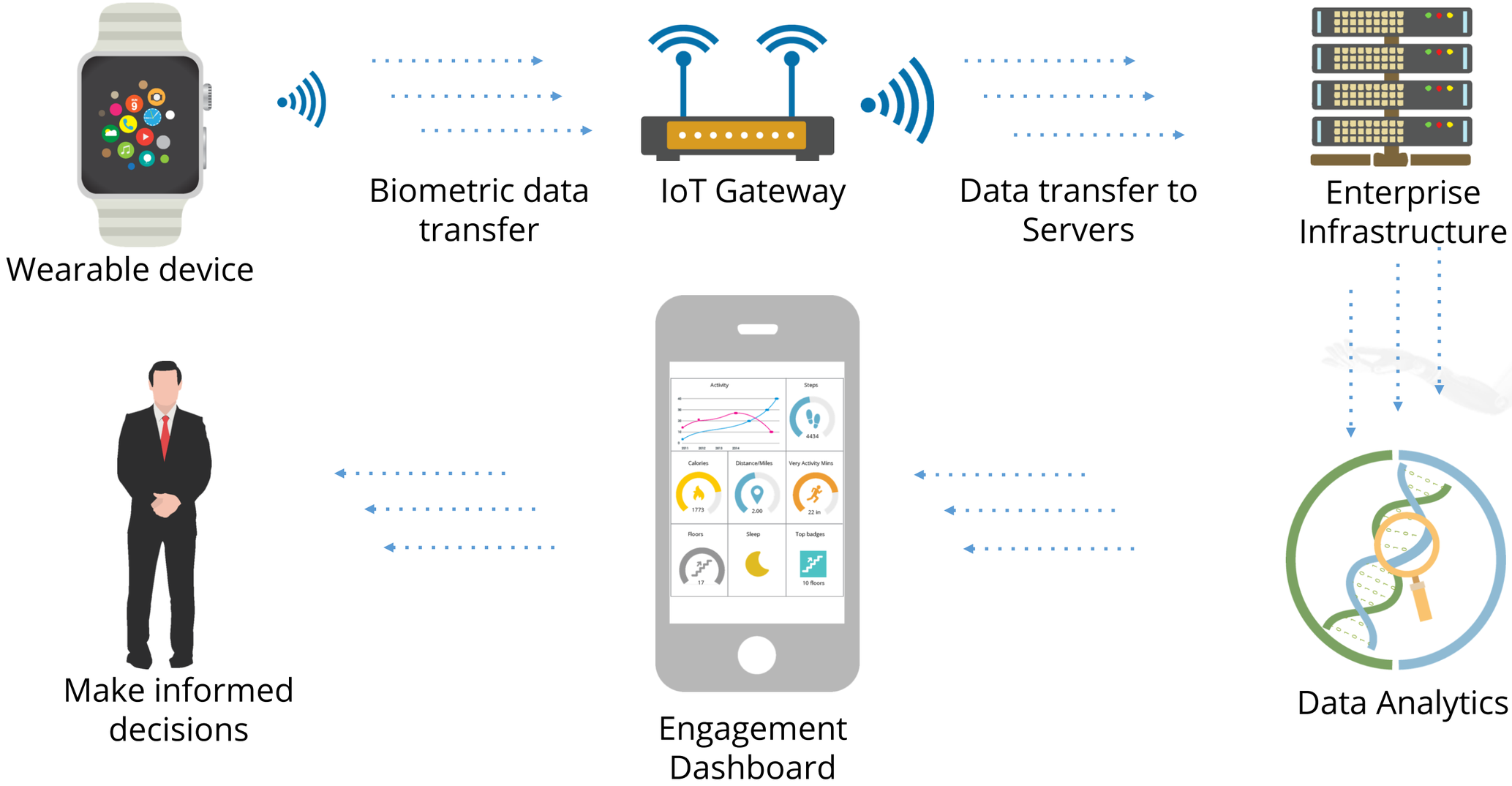
Autocomplete feature is fed by data models (Machine Learning)

Fast and real-time analytics is made possible by modern and advanced infrastructure, tools, and technologies

Influencing Factors
1. Query Volume – Unique and verifiable users 2. Geographical locations 3. Keyword/phrase matches on the web 4. Some scrubbing for inappropriate content

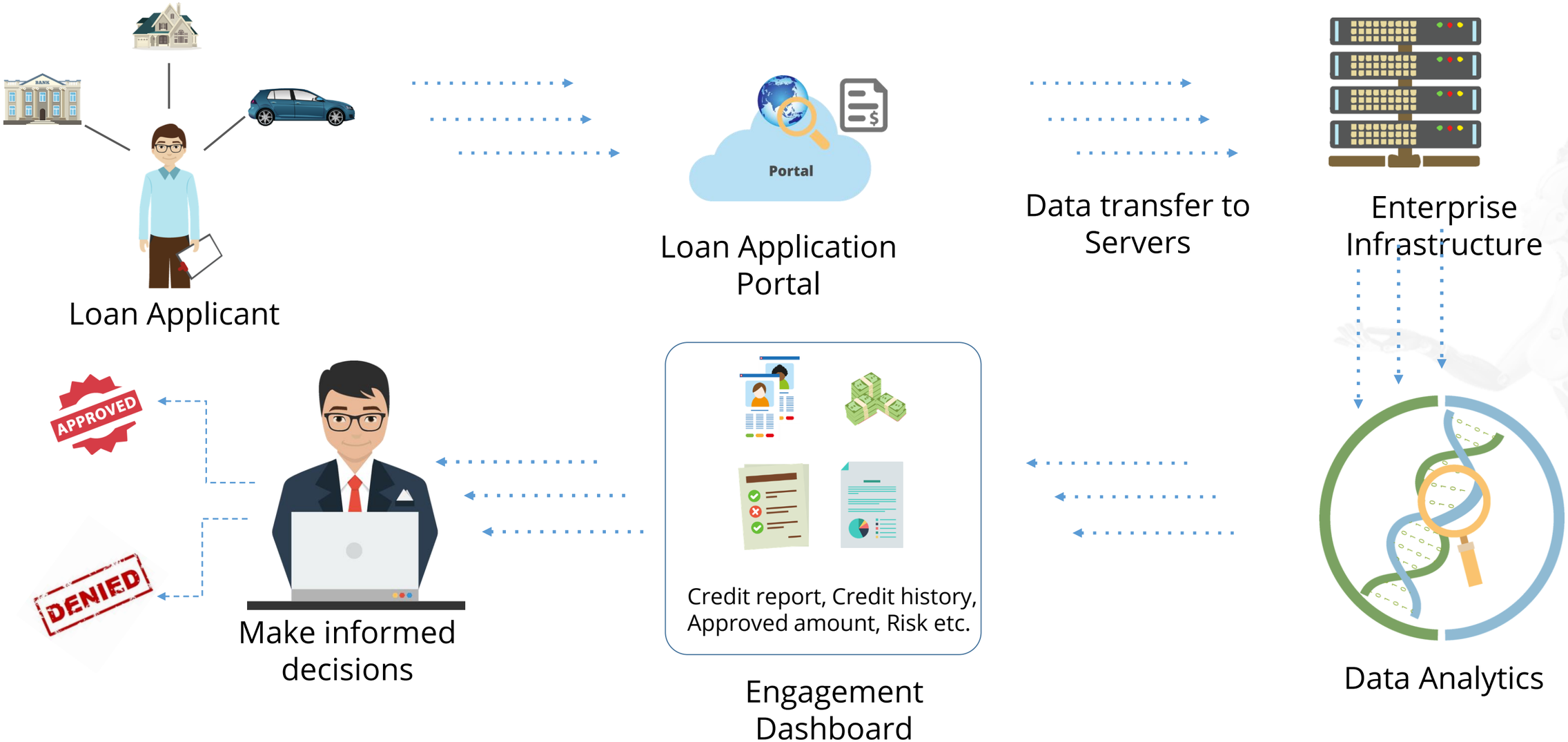
Using Data Science: Healthcare

Wearable devices use Data Science to analyze data gathered by their biometric sensors.



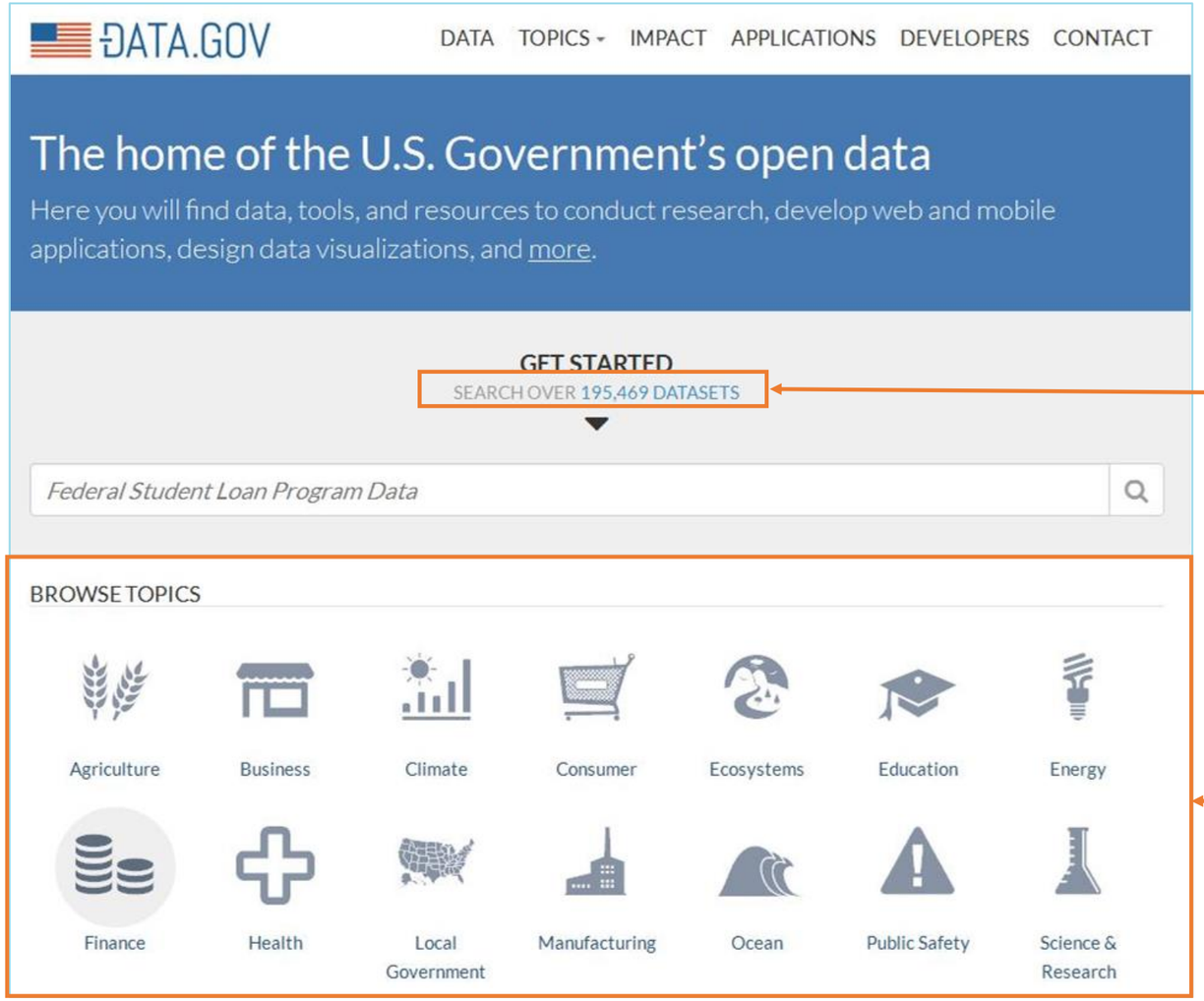
Using Data Science: Finance

A loan manager can easily access and sift through a loan applicant's financial details using Data Science.



Using Data Science: Public Sector

The governments in different countries share large datasets from various domains with the public. Data.gov is a website hosted and maintained by the U.S. government.



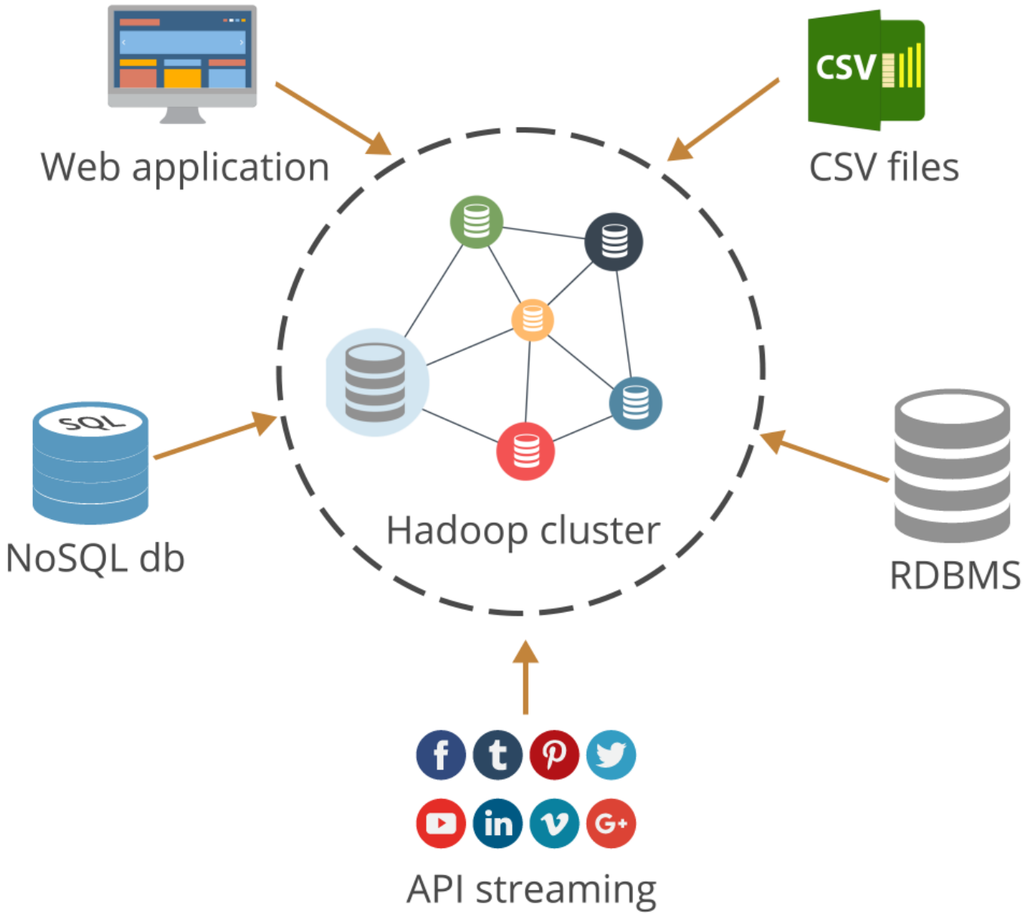
Large collection of datasets

Sectors/Domains

The Real Challenge

Some of the challenges Data Scientists face in the real world are:

- Data quality doesn't conform to the set standards
- Data integration is a complex task
- Data is distributed into large clusters in HDFS, which is difficult to integrate and analyze
- Unstructured and semi-structured data are harder to analyze



Data Analytics and Python

Python deals with each stage of data analytics efficiently by applying different libraries and packages.

Data
Analytics

Acquire	
Wrangle	pandas
Explore	matplotlib
Model	scikit learn, NumPy
Visualize	Bokeh, SciPy

Python Tools and Technologies

Python is a general purpose, open source programming language that lets you work quickly and integrate systems more effectively.

Data analysis

Pandas

Scientific computing

NumPy

Machine learning

Learn

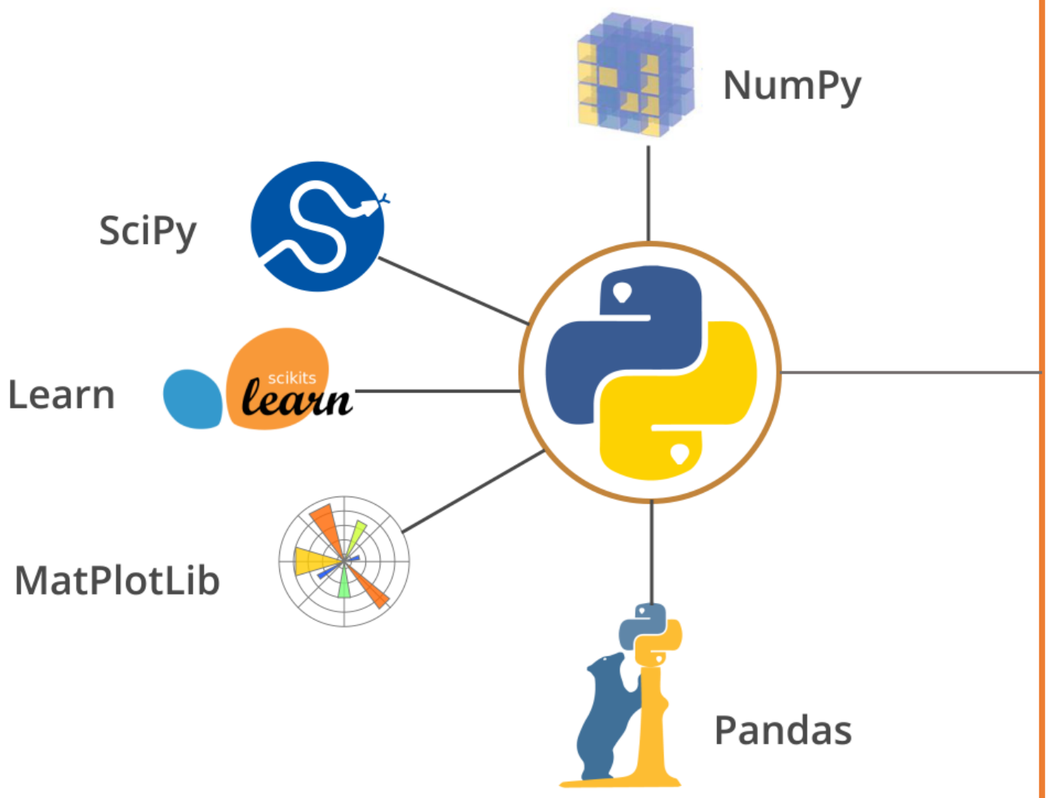
Software for mathematics, science, and engineering

SciPy

Graphics computing

MatPlotLib

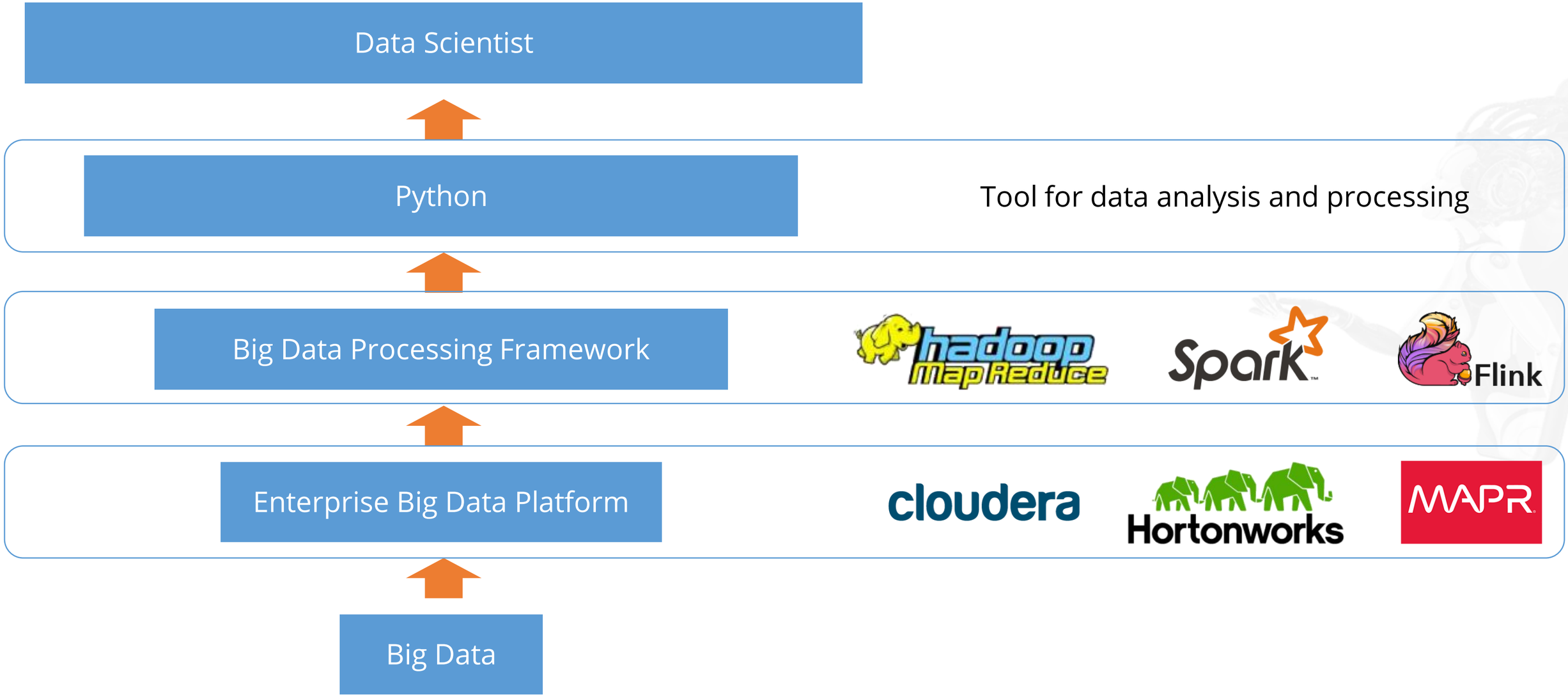
Benefits of Python



- Easy to learn
- Open source
- Efficient and multi-platform support
- Huge collection of libraries, functions, and modules
- Big open source community
- Integrates well with enterprise apps and systems
- Great vendor and product support

Big Data Platforms and Processing Frameworks for Python

Python is supported by well-established data platforms and processing frameworks that help analyze data in a simple and efficient way.



Knowledge Check

Knowledge Check 1	A Data Scientist _____.

a. Asks the right questions

b. Acquires data

c. Performs data wrangling and data visualization

d. All of the above

Knowledge Check 1	A Data Scientist _____.

a. Asks the right questions

b. Acquires data

c. Performs data wrangling and data visualization

d. All of the above

The correct answer is **d**

A Data Scientist asks the right questions to the stakeholders, acquires data from various sources and data points, performs data wrangling that makes the data available for analysis, and creates reports and plots for data visualization.

Knowledge Check 2	**The search engine's autocomplete feature identifies unique and verifiable users who search for a particular keyword or phrase to_____. Select all that apply.**

a. Scrub inappropriate content

b. Build a query volume

c. Tag the location to a query

d. Find similar instances on the web

Knowledge Check 2	**The search engine's autocomplete feature identifies unique and verifiable users who search for a particular keyword or phrase to_____. Select all that apply.**

a. Scrub inappropriate content

b. Build a query volume

c. Tag the location to a query

d. Find similar instances on the web

The correct answer is **b, c**

The search engine's autocomplete feature identifies unique and verifiable users who search for a particular keyword or phrase to build a query volume. It also helps identify the users' locations and tag them to the query, enabling it to be location-specific.

Knowledge Check 3	What is the sequential flow of Data Analytics?

a. Data wrangling, exploration, modeling, acquisition, and visualization

b. Data exploration, acquisition, modeling, wrangling, and visualization

c. Data acquisition, wrangling, exploration, modeling, and visualization

d. Data modeling, acquisition, exploration, wrangling, and visualization

Knowledge Check 3	What is the sequential flow of Data Analytics?

a. Data wrangling, exploration, modeling, acquisition, and visualization

b. Data exploration, acquisition, modeling, wrangling, and visualization

c. Data acquisition, wrangling, exploration, modeling, and visualization

d. Data modeling, acquisition, exploration, wrangling, and visualization

The correct answer is **c**

In Data Analytics, the data is acquired from various sources and is then wrangled to ease its analysis. This is followed by data exploration and data modeling. The final stage is data visualization, where the data is presented and the patterns are identified.

Key Takeaways

You are now able to:

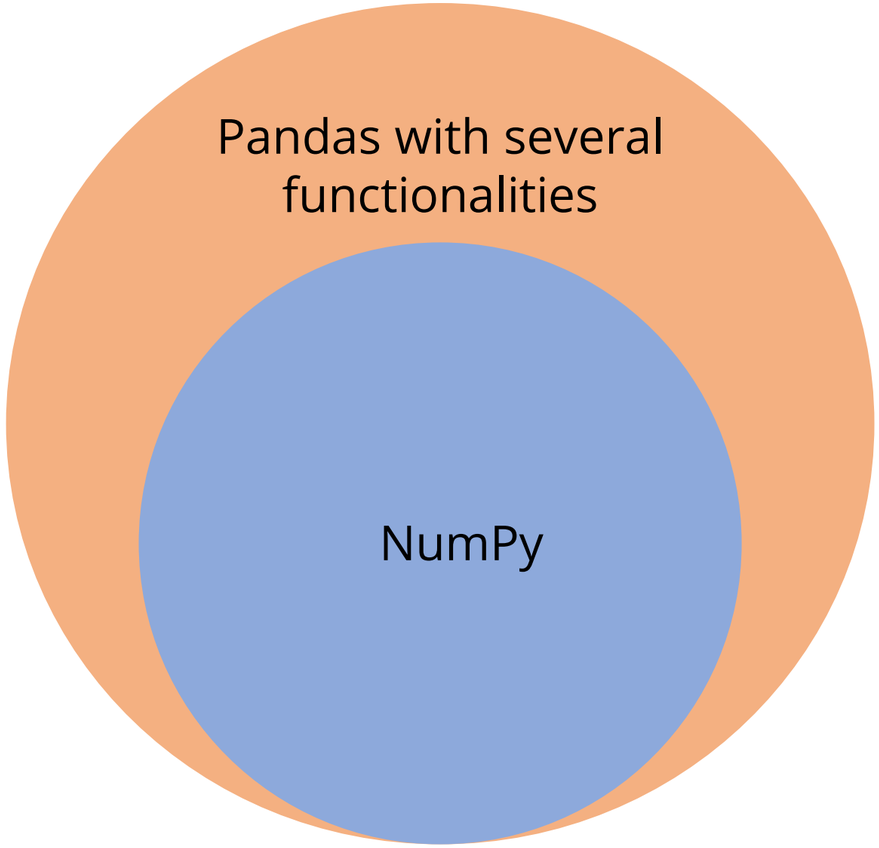
- Define Data Science
- Discuss the roles and responsibilities of a Data Scientist
- List various applications of Data Science
- Explain Data Science importance
- Describe Python and its importance

Introduction to Pandas

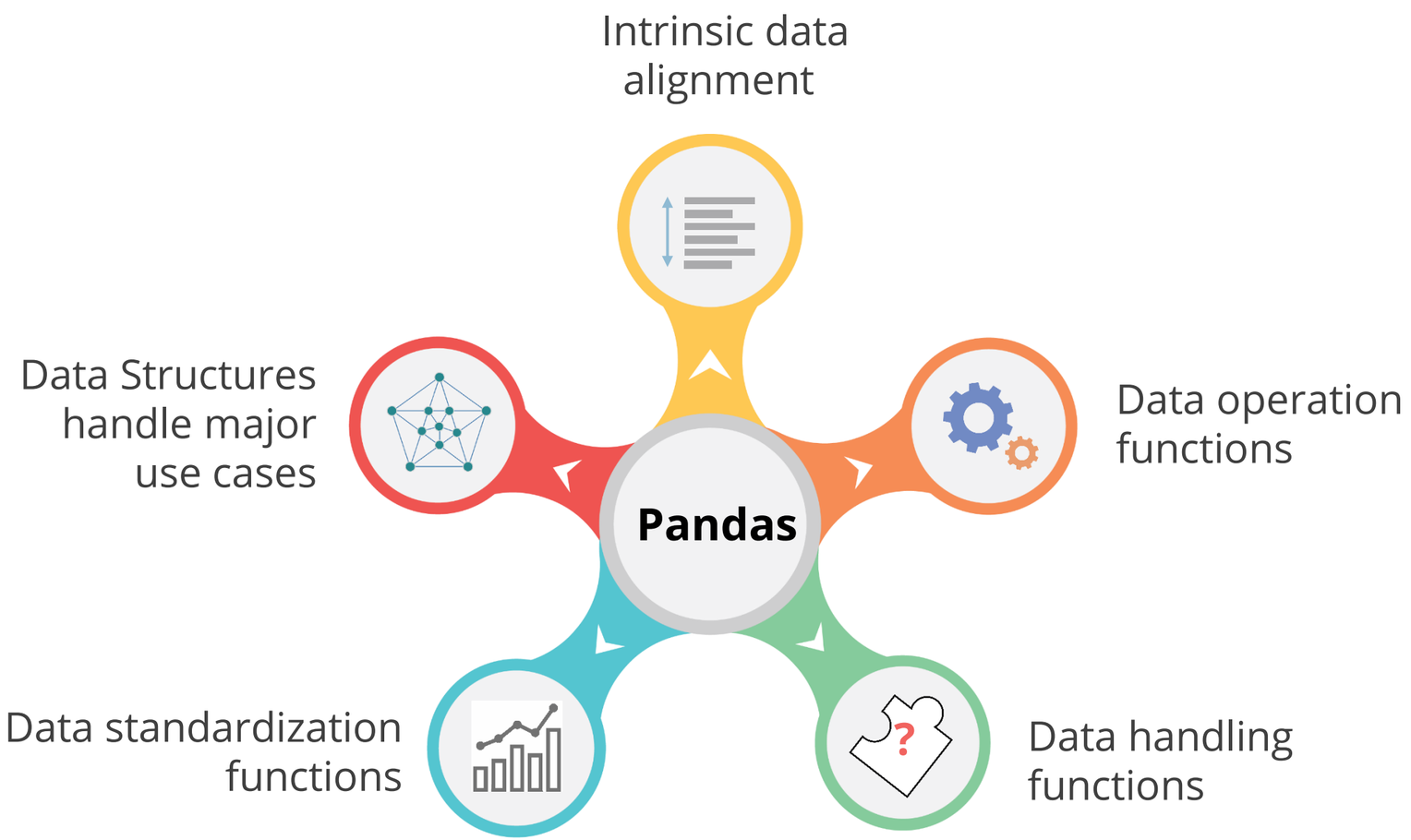
©Simplilearn. All rights reserved.

Why Pandas

NumPy is great for mathematical computing, but why do we need Pandas?

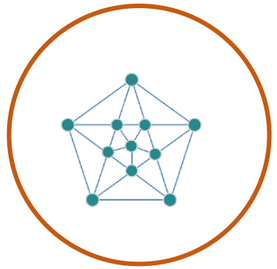


Why Pandas


Intrinsic data alignment
Data Structures handle major use cases
Pandas
Data operation functions
Data standardization functions
Data handling functions

Why Pandas

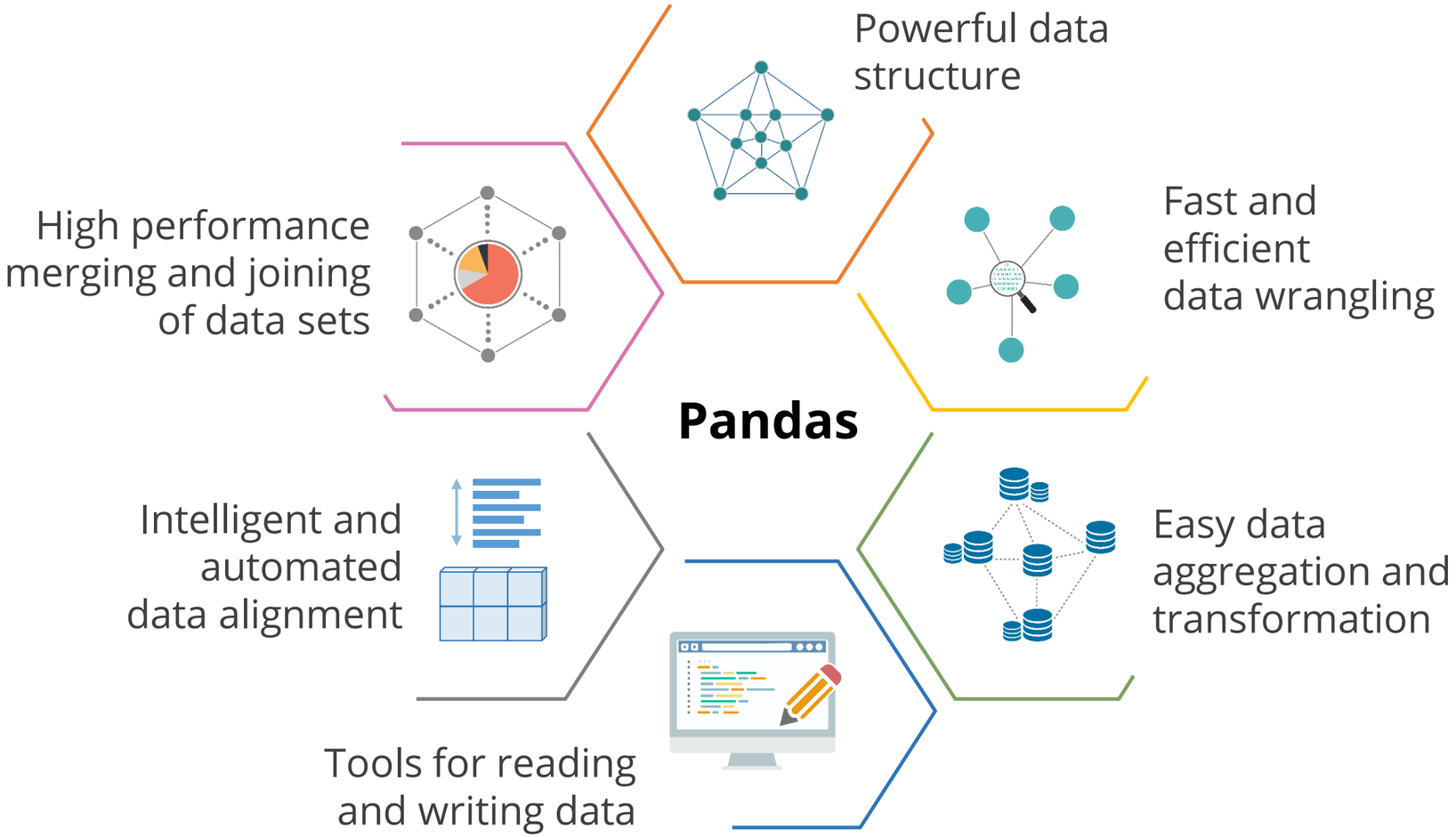
Data structures handling major use cases



Pandas

Features of Pandas

The various features of Pandas make it an efficient library for Data Scientists.



Data Structures

The four main libraries of Pandas data structure are:

Series

- **One**-dimensional labeled array
- Supports multiple data types

Data Frame

- **Two**-dimensional labeled array
- Supports multiple data types
- Input can be a series
- Input can be another DataFrame

Panel

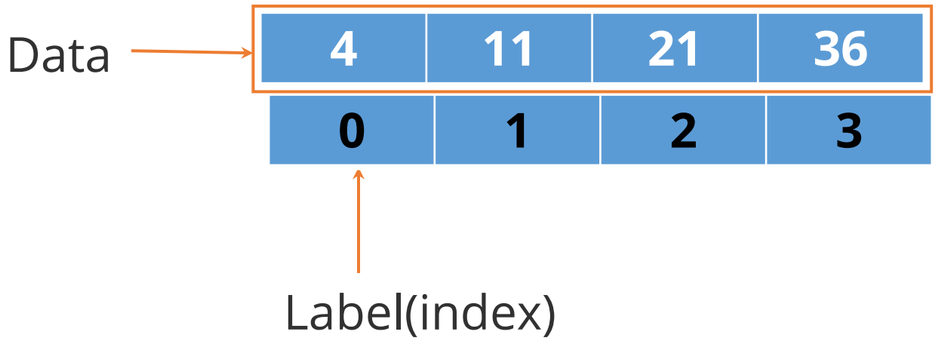
- **Three**-dimensional labeled array
- Supports multiple data types
- Items axis 0
- Major axis rows
- Minor axis columns

Panel 4D (Experimental)

- **Four-**dimensional labeled array
- Supports multiple data types
- Labels axis 0
- Items axis 1
- Major axis rows
- Minor axis columns

Understanding Series

Series is a one-dimensional array-like object containing data and labels (or index).



Data alignment is intrinsic and will not be broken until changed explicitly by program.

Series

Series can be created with different data inputs:

Data Types

- Integer
- String
- Python Object
- Floating Point

Data Input

- ndarray
- dict
- scalar
- list

Series

2	3	8	4
0	1	2	3

Label(index)

How to Create Series?

Key points to note while creating a series are:

- Import Pandas as it is the main library (Import pandas as pd)
- Import NumPy while working with ndarrays (Import numpy as np)
- Apply the syntax and pass the data elements as arguments

Basic Method

```
S = pd.Series(data, index = [index])
```

4	11	21	36

Series

Creating Series from a List

```
In [14]: import numpy as np
         import pandas as pd
```

Import libraries

```
In [15]: first_series = pd.Series(list('abcdef'))
```

Pass list as an argument

```
In [16]: print (first_series)
```

```
0    a
1    b
2    c
3    d
4    e
5    f
dtype: object
```

Index

Data value

Data type

We have not created index for data but notice that data alignment is done automatically.

Creating Series from an ndarray

ndarray for countries

```
In [17]: np_country = np.array(['Luxembourg','Norway','Japan','Switzerland','United States','Qatar','Iceland','Sweden',
                                'Singapore','Denmark'])
```

```
In [18]: s_country = pd.Series(np_country)
```

Pass ndarray as an argument

```
In [19]: print (s_country)
```

```
0       Luxembourg
1           Norway
2            Japan
3      Switzerland
4    United States
5            Qatar
6          Iceland
7           Sweden
8        Singapore
9          Denmark
dtype: object
```

countries

Data type

Creating Series from dict

A series can also be created with dict data input for faster operations.

dict for countries and their gdp

```
In [10]: #Evaluate countries and their corresponding gdp per capita and print them as series
         dict_country_gdp = pd.Series([52056.01781,40258.80862,40034.85063,39578.07441,39170.41371,37958.23146,37691.02733,
                                       36152.66676,34706.19047,33630.24604,33529.83052,30860.12808],index=['Luxembourg','Macao, China','Norway',
                                  'Japan','Switzerland','Hong Kong, China','United States','Qatar','Iceland','Sweden','Singapore','Denmark'])
```

Countries have been passed as an index and GDP as the actual data value

```
In [11]: print (dict_country_gdp)
```

```
Luxembourg          52056.01781
Macao, China        40258.80862
Norway              40034.85063
Japan               39578.07441
Switzerland         39170.41371
Hong Kong, China    37958.23146
United States       37691.02733
Qatar               36152.66676
Iceland             34706.19047
Sweden              33630.24604
Singapore           33529.83052
Denmark             30860.12808
dtype: float64
```

GDP

Country

Data type

Creating Series from Scalar

Scalar input

```
In [31]: #Print Series with scalar input
         scalar_series = pd.Series(5.,index=['a','b','c','d','e'])
```

```
In [32]: scalar_series
```

Index

```
Out[32]: a    5
         b    5
         c    5
         d    5
         e    5
         dtype: float64
```

Data

index

Data type

Accessing Elements in Series

Data can be accessed through different functions like loc, iloc by passing data element position or index range.

```
In [43]: #access elements in the series
         dict_country_gdp[0]

Out[43]: 52056.017809999998

In [44]: #access first 5 countries from the series
         dict_country_gdp[0:5]

Out[44]: Luxembourg      52056.01781
         Macao, China    40258.80862
         Norway          40034.85063
         Japan           39578.07441
         Switzerland     39170.41371
         dtype: float64

In [45]: #look up a country by name or index
         dict_country_gdp.loc['United States']

Out[45]: 37691.027329999997

In [46]: #look up by position
         dict_country_gdp.iloc[0]

Out[46]: 52056.017809999998
```

Vectorizing Operations in Series

Vectorized operations are performed by the data element's position.

Add the series

```
In [52]: first_vector_series = pd.Series([1,2,3,4],index=['a','b','c','d'])
         second_vector_series = pd.Series([10,20,30,40],index=['a','b','c','d'])

In [53]: first_vector_series+second_vector_series

Out[53]: a    11
         b    22
         c    33
         d    44
         dtype: int64

In [54]: second_vector_series = pd.Series([10,20,30,40],index=['a','d','b','c'])

In [55]: first_vector_series+second_vector_series

Out[55]: a    11
         b    32
         c    43
         d    24
         dtype: int64
```

Vectorizing Operations in Series

```
In [19]: #now replace few indexes with new ones in second vector series
         second_vector_series = pd.Series([10,20,30,40],index=['a','b','e','f'])

In [20]: first_vector_series+second_vector_series

Out[20]: a    11
         b    22
         c   NaN
         d   NaN
         e   NaN
         f   NaN
         dtype: float64
```

DataFrames

DataFrame

DataFrame is a two-dimensional labeled data structure with columns of potentially different types.

Data Types

- Integer
- String
- Python Object
- Floating Point

Data Input

- ndarray
- dict
- List
- Series
- DataFrame

2	3	8	4
5	8	10	1
0	1	2	3

Label(index)

DataFrame

©Simplilearn. All rights reserved.

simplilearn

Creating DataFrame from Lists

```
In [1]: import pandas as pd
```

Create DataFrame from dict of equal length lists

```
In [2]: #last five olymnics data: place, year and number of countries participated
        olympic_data_list = {'HostCity':['London','Beijing','Athens','Sydney','Atlanta'],
                             'Year':[2012,2008,2004,2000,1996],
                              'No. of Participating Countries':[205,204,201,200,197]
                             }
```

```
In [3]: df_olympic_data = pd.DataFrame(olympic_data_list)
```

Pass the list to the DataFrame

```
In [4]: df_olympic_data
```

Out[4]:

	HostCity	No. of Participating Countries	Year
0	London	205	2012
1	Beijing	204	2008
2	Athens	201	2004
3	Sydney	200	2000
4	Atlanta	197	1996

Creating DataFrame from dict

This example shows you how to create a DataFrame from a series of dicts.

dict one

dict two

Create DataFrame from dict of dicts

```
In [5]: olympic_data_dict = {'London':{2012:205},'Beijing':{2008:204}}

In [6]: df_olympic_data_dict = pd.DataFrame(olympic_data_dict)

In [7]: df_olympic_data_dict
```

Entire dict

Out[7]:

	Beijing	London
2008	204	NaN
2012	NaN	205

Viewing DataFrame

You can view a DataFrame by referring to the column name or with the describe function.

```
In [8]: #select by City name
        df_olympic_data.HostCity
```

```
Out[8]: 0     London
        1    Beijing
        2     Athens
        3     Sydney
        4    Atlanta
        Name: HostCity, dtype: object
```

```
In [9]: #use describe function to display the content
        df_olympic_data.describe
```

```
Out[9]: <bound method DataFrame.describe of   HostCity  No. of Participating Countries  Year
        0   London                            205  2012
        1  Beijing                            204  2008
        2   Athens                            201  2004
        3   Sydney                            200  2000
        4  Atlanta                            197  1996>
```

Creating DataFrame from dict of Series

Create DataFrame from dict of series

In [10]:
```
olympic_series_participation = pd.Series([205,204,201,200,197],index=[2012,2008,2004,2000,1996])
olympic_series_country = pd.Series(['London','Beijing','Athens','Sydney','Atlanta'],
                                   index=[2012,2008,2004,2000,1996])
```

In [11]:
```
df_olympic_series = pd.DataFrame({'No. of Participating Countries':olympic_series_participation,
                                  'Host Cities':olympic_series_country})
```

In [12]:
```
df_olympic_series
```

Out[12]:

	Host Cities	No. of Participating Countries
2012	London	205
2008	Beijing	204
2004	Athens	201
2000	Sydney	200
1996	Atlanta	197

Creating DataFrame from ndarray

Create DataFrame from dict of ndarray

```
In [13]: import numpy as np

In [14]: np_array = np.array([2012,2008,2004,2006])
         dict_ndarray = {'year':np_array}

In [15]: df_ndarray = pd.DataFrame(dict_ndarray)

In [16]: df_ndarray
```

Create a ndarray with years

Create a dict with the ndarray

Pass this dict to a new DataFrame

Out[16]:

	year
0	2012
1	2008
2	2004
3	2006

Creating DataFrame from DataFrame Object

In [17]:
```
df_from_df = pd.DataFrame(df_olympic_series)
```

Create a DataFrame from a DataFrame object

In [18]:
```
df_from_df
```

Out[18]:

	Host Cities	No. of Participating Countries
2012	London	205
2008	Beijing	204
2004	Athens	201
2000	Sydney	200
1996	Atlanta	197